Support vector machines

A focused six-page guide: scores, margins, kernels, and decisions

1. A linear score and its margin

A **support vector machine (SVM)** learns a classification boundary while controlling its complexity. Start with binary labels $y_i \in \{-1, +1\}$ and input vectors $x_i \in \mathbb{R}^p$. The *score* is $f(x) = w^T x + b$: w is the slope vector and b the intercept. We predict $+1$ when $f(x) \geq 0$, otherwise -1, explicitly choosing $+1$ at a tie. The boundary $f(x) = 0$ is a line in two dimensions and a hyperplane in higher dimensions. **A score is not a probability**: it can be negative or larger than one.

Two meanings of margin. For $w \neq 0$, define

$$\boxed{\text{functional margin } m_i = y_i(w^T x_i + b), \qquad \text{signed geometric margin } \frac{m_i}{\|w\|}.}$$

A positive functional margin means the point is on the correct side; a negative one means the wrong side. Zero lies on the boundary, where correctness depends on the chosen tie convention. Distance to the boundary is $|f(x)|/\|w\|$. Multiplying (w, b) by any positive number leaves the boundary and geometric margins unchanged, but changes functional margins. We need a normalization before maximizing a functional quantity.

Hard-margin normalization. If the classes are strictly linearly separable, require $m_i \geq 1$ and solve

$$\boxed{\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to } y_i(w^T x_i + b) \geq 1 \ \forall i.}$$

The parallel supporting planes $f(x) = \pm 1$ are each $1/\|w\|$ from the boundary, so the full slab width is $2/\|w\|$. Minimizing norm maximizes this margin. The intercept is not penalized. Nonseparable data have no feasible hard-margin solution.

A solution we can verify. Take negative points $(-3, 1), (-1, 0)$ and positive points $(1, -1), (1, 1), (3, 0)$. The solution is $w = (1, 0)$, $b = 0$. Its margin is 1, width 2, and objective $1/2$. The negative point $(-1, 0)$ implies $w_1 - b \geq 1$; the mean of the two positive constraints at $(1, \pm 1)$ implies $w_1 + b \geq 1$. Thus $w_1 \geq 1$, so no feasible vector has norm below 1. This proves optimality, not just separation.

Remember. A perfectly separated training set does not ensure good prediction. Outliers, overlap, and feature units matter. Soft margins allow violations instead of requiring perfect separation.

2. Soft margins, hinge loss, and the meaning of C

Introduce nonnegative slack variables ξ_i to allow violations of the unit margin:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i, \qquad y_i f(x_i) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad C > 0.$$

For fixed scores, the smallest permissible slack is $\xi_i = \max(0, 1 - y_i f(x_i))$. This is the **hinge loss**, so the equivalent objective is

$$\boxed{\frac{1}{2}\|w\|^2 + C\sum_i \max(0, 1 - y_i f(x_i)).}$$

Functional margin m	Hinge loss	Interpretation
$m \geq 1$	0	Correct side, unit margin met
$0 < m < 1$	$1 - m$	Correct label, inside margin
$m < 0$	$1 - m > 1$	Wrong side of boundary

At $m = 0$, loss is 1 and the tie rule decides the label. A slack is not a probability or a geometric distance. For margins $(1.4, 0.3, -0.2)$, hinge losses are $(0, 0.7, 1.2)$: two points have positive loss, but only the last is strictly on the wrong side.

C is a tradeoff. Larger C charges more for margin violations relative to coefficient norm. Smaller C favors a smaller norm, allowing more violations. It is not a maximum error count, and no finite setting guarantees a particular test accuracy. For n fitting rows, divide by nC to obtain

$$\frac{1}{n}\sum_i \max(0, 1 - y_i f_i) + \frac{\lambda}{2}\|w\|^2, \qquad \boxed{\lambda = \frac{1}{nC}}.$$

A conversion between libraries must match sample-size normalization, feature scaling, hinge versus squared-hinge loss, and intercept treatment. Fixing C across folds is a stated tuning convention; it does not fix the equivalent mean-loss λ across different fitting sizes.

A small soft-margin calculation. For points $(-1, -1)$ and $(1, +1)$, one optimum has $b = 0$ and $w = \min(2C, 1)$. At $C = 0.2$, $w = 0.4$: both labels are correct but each slack is 0.6. The objective is $0.4^2/2 + 0.2(1.2) = 0.32$.

A subgradient for implementation. For the mean-loss objective, let $A = \{i : y_i f_i < 1\}$. Away from margin 1,

$$g_w = \lambda w - \frac{1}{n}\sum_{i \in A} y_i x_i, \qquad g_b = -\frac{1}{n}\sum_{i \in A} y_i.$$

At margin 1 the derivative is not unique; zero is one valid contribution for that row's hinge term. Finite-difference checks should avoid such corners. Our focused notebooks verify this formula and let `SVC` optimize the model rather than introducing a second optimizer to tune.

3. Support vectors explain the fitted rule

The binary soft-margin dual assigns one nonnegative coefficient α_i to each fitting row:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j, \quad 0 \le \alpha_i \le C, \quad \sum_i \alpha_i y_i = 0.$$

For an optimum, $w = \sum_i \alpha_i y_i x_i$. Therefore

$$\boxed{f(x) = \sum_{i \in SV} \alpha_i y_i \, x_i^T x + b, \qquad SV = \{i : \alpha_i > 0\}.}$$

These selected fitting observations are **support vectors**. They are not a special test set, and they are not exactly the misclassified points. The optimality conditions imply:

Dual coefficient	Functional margin at an exact optimum
$\alpha_i = 0$	$y_i f_i \ge 1$
$0 < \alpha_i < C$	$y_i f_i = 1$
$\alpha_i = C$	$y_i f_i \le 1$

Degenerate points can lie exactly on the margin without a positive coefficient. Numerical solvers use tolerances, so do not demand exact floating-point equalities when interpreting a fit. For a free support vector, $b = y_i - w^T x_i$; different free support vectors agree at the exact optimum.

Certify the five-point example. In the order on page 1, choose $\alpha = (0, 1/2, 1/4, 1/4, 0)$. It satisfies nonnegativity and $\alpha^T y = 0$, reconstructs $w = (1, 0)$, and gives dual value $1 - 1/2 = 1/2$. Feasibility plus equal primal and dual values certifies optimality. The same coefficients satisfy the soft-margin upper bounds when $C \ge 1/2$. Application A checks these facts numerically with a comfortably larger reference C.

Python objects. `SVC(kernel="linear", C=1)` creates a classifier; Its `fit` method learns from X and y; `decision_function` returns scores for new inputs. `support_` stores fitting-row positions, `support_vectors_` stores their input coordinates, and `n_support_` counts them by class. `dual_coef_` contains the signed coefficients $\alpha_i y_i$ in binary classification. For a linear kernel, `coef_` and `intercept_` give w and b.

Scaling changes the geometry. With $z_j = (x_j - \mu_j)/a_j$, margins refer to standardized space. If the fitted standardized parameters are $(\tilde{w}, \tilde{b})$, then $w_j = \tilde{w}_j / a_j$ and $b = \tilde{b} - \sum_j w_j \mu_j$ express the same score in original units. The maximum-margin interpretation still uses the geometry in which the model was trained.

Remember. `LinearSVC` is not a drop-in numerical identity check for `SVC(kernel="linear")`: its default loss and intercept handling differ. Match the objectives before comparing coefficients.

4. Kernels turn linear feature-space rules into curved boundaries

For a feature map ϕ, a kernel is $K(x, z) = \phi(x)^T \phi(z)$. Replace inner products in the dual and prediction rule:

$$\boxed{f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b.}$$

A valid real kernel has symmetric positive semidefinite Gram matrices: for any sample, $v^T K v \geq 0$ for every real vector v. An arbitrary similarity function need not satisfy this.

An exact quadratic example. For $x = (x_1, x_2)$,

$$K(x, z) = (1 + x^T z)^2, \qquad \phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1 x_2, x_2^2).$$

Expanding the dot product verifies the identity. A circle such as $x_1^2 + x_2^2 = 1.5$ is a linear equation in squared coordinates but not in the two original inputs. An explicit-feature linear SVM and this polynomial-kernel SVM solve the same problem when scaling, C, loss, and intercept conventions match. Omitting the square-root factors changes the penalty geometry. Apply input scaling before both maps, without independently rescaling the expanded columns.

RBF means radial basis function. The Gaussian RBF kernel is

$$\boxed{K(x, z) = \exp(-\gamma \|x - z\|^2), \qquad \gamma > 0.}$$

At distance $1/\sqrt{\gamma}$, similarity is e^{-1}. Small gamma gives broad similarities; large gamma gives very local similarities and can fit fine detail or noise. C and gamma act together, so select them on a joint grid rather than deciding from one training plot.

Why feature units matter. Changing only x_1 to $100x_1$ changes squared distance to $10000(x_1 - z_1)^2 + (x_2 - z_2)^2$. One scalar gamma cannot generally undo this anisotropic change. Training-based standardization prevents an arbitrary unit convention from dominating distances. Use `make_pipeline(StandardScaler(), SVC(...))` so that every fitting fold learns its own scale.

Useful options. `kernel="rbf"` selects the RBF; `gamma` controls locality. For `kernel="poly"`, the kernel is $(\gamma x^T z + \text{coef0})^{\text{degree}}$. Thus `degree=2, gamma=1, coef0=1` implements the quadratic kernel above. The kernel's `coef0` is not the classifier's fitted intercept.

Computational cost. A full $n \times n$ kernel matrix has quadratic storage, although solvers can cache only parts. Prediction usually needs similarities to the support vectors; more support vectors can increase latency. Kernel SVMs are useful on moderate data sizes; a linear solver can be more practical for very large sparse feature matrices.

5. Select a model, then select a decision policy

Data roles. Training learns preprocessing and coefficients. Five-fold cross-validation (CV) repeatedly fits on four training groups and scores the fifth. Use the same folds for competing settings, and fit every scaler inside each fold. Validation can select a decision threshold or audit frozen choices. The final test is reserved for assessment, not another tuning round.

Accuracy is the fraction of correct decisions. With a designated positive class, let TP and FN count correctly detected and missed positives, and TN and FP correctly rejected and falsely predicted positives. Then

$$\text{balanced accuracy} = \tfrac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right),$$
$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}.$$

Balanced accuracy weights classes equally. Undefined ratios need an explicit reporting convention. The ROC AUC (area under the receiver operating characteristic curve) assesses score ranking; it does not require calibrated probabilities and is unchanged by a strictly increasing score transformation.

Scores are not risks. In binary `SVC`, check `classes_`: positive scores favor the second class. For OJ, encode CH as 0 and MM as 1. A score of 0.25 does not mean a 25% chance of MM. The default zero cutoff is a classification convention, not necessarily a cost-minimizing policy.

Suppose false positives cost 1 and false negatives cost 3. On m validation rows, minimize

$$\boxed{\text{mean cost}(t) = \frac{FP(t) + 3FN(t)}{m}, \qquad \hat{y}(t) = \mathbf{1}\{f(x) \geq t\}.}$$

Sorted distinct scores determine all decision patterns, including always positive and always negative. Specify a tie rule; our notebook chooses the largest cutoff among equal-cost policies. A finite validation optimum can vary substantially between samples.

If p were a true conditional probability, expected positive-decision cost would be $1 \times (1-p)$, and negative-decision cost $3 \times p$. The ideal probability threshold would be $1/4$. **This derivation does not make** $1/4$ **a raw SVM score threshold.** Do not apply a sigmoid to scores and assume the result is calibrated. Calibration learns an additional mapping on appropriately separated development data and must itself be assessed.

Class weighting is different. Weighting a class changes the training penalties and can change the fitted boundary and scores. Changing the prediction threshold leaves the fitted score function intact. Both are possible modeling decisions, but they are not interchangeable implementation switches.

Remember. Freeze preprocessing, model, and threshold before final testing. After a threshold is selected, refitting the model may change the score scale and invalidate that threshold. Random record splits do not establish future-time or new-group performance when records share stores, users, or time periods.

6. Applications, exercises, and a practical checklist

A: margins and hinge loss. Verify the five-point solution and its dual certificate; check a hinge subgradient away from its corner; fit scaled linear SVMs on 400 overlapping Gaussian examples; inspect one conflicting training observation. CV chooses C, while a separate perturbation is diagnostic only. The output includes support counts, a margin distribution, and a frozen test comparison.

B: kernels and scaling. Use 1,200 uniform points with a circular clean boundary and 8% random label flips. Verify the explicit quadratic map against the matching kernel. Compare logistic, linear, quadratic, polynomial, and RBF models using training CV balanced accuracy. Plot the RBF grid and boundaries; change one input's units by a factor of 100. Score the noisy observed labels, not an easier clean target substituted after fitting.

C: OJ score thresholds. Load 1,070 textbook purchases directly from a URL. Predict MM using six recorded price/discount/promotion features. Tune logistic and SVM comparisons, select an SVM using CV, and choose its score cutoff under costs 1 and 3 using validation. Report costs as well as accuracy and ranking. Store and week diagnostics make the scope of the random-split result explicit.

D: short student worksheet. Complete five one- or two-line gaps: fit a scaled linear SVM, predict labels, fit a scaled RBF SVM for two rings, compute scores, and turn scores into decisions. Data generation and evaluation are supplied. The answer key includes completed code, explanations, and checked results.

Check your understanding.

1. For $w = (3, 4)$, $b = -2$, and $x = (2, 1)$, find the score and distance to the boundary.
2. For margins $(-0.5, 0.4, 1.3)$, find hinge losses and identify correctly classified points with positive loss.
3. If $n = 200$ and $C = 2$, what is the matching mean-loss λ?
4. For RBF gamma 4, at what Euclidean distance is similarity e^{-1}?
5. Why is a score threshold selected from validation not necessarily zero? Why is it not automatically 1/4?

Answers: (1) score 8, distance 8/5; (2) losses $(1.5, 0.6, 0)$, middle point; (3) 1/400; (4) 1/2; (5) costs and score distributions matter, and scores are not probabilities.

Before trusting a result: check label order; keep targets out of features; learn scales within fitting folds; state loss and C conventions; distinguish support vectors from errors; examine held-out scores; report the chosen metric and threshold; and preserve the fitted pipeline used to select that threshold.

Documentation. The notebook API choices follow the course's scikit-learn 1.8 environment. See the official SVM guide and textbook OJ data description. The longer course chapter gives full derivations, optimization extensions, multiclass methods, and support vector regression. Worked PDF explanations and the student answer key are in `FocusedContent/Solutions`.